TP N°3 :

Introduction aux assertions

Exercice 1 :

1) Écrire un sous-programme (en utilisant le langage C ou Java) d'une recherche de la position de la valeur maximale d'un tableau d'entier.
2) Définir un variant et un invariant de boucle dans la solution proposée.
3) Étendre ces sous programme par des pré et post conditions puis tester le programme.

Exercice 2:

La technique de recherche dichotomique n'est applicable que si le tableau est **déjà trié** (par exemple dans l'ordre croissant). Le but de recherche dichotomique est de diviser l'intervalle de recherche par 2 à chaque itération. Pour cela, on procède de la façon suivante :
Soient premier et dernier les extrémités gauche et droite de l'intervalle dans lequel on cherche la valeur x, on calcule M, l'indice de l'élément médian :
M = (premier + dernier) div 2
Il y a 3 cas possibles :

- x = T[M] : l'élément de valeur x est trouvé, la recherche est terminée
- x < T[M] : l'élément x, s'il existe, se trouve dans l'intervalle [premier..M-1]
- x >T[M] : l'élément x, s'il existe, se trouve dans l'intervalle [M+1..dernier]

La recherche dichotomique consiste à itérer ce processus jusqu'à ce que l'on trouve x ou que l'intervalle de recherche soit vide.

1) Réaliser en C ou en Java la fonction **int RechDichotomique (int T [] , int n, int info)**
2) Définir un variant et un invariant de boucle dans la solution proposée.
3) Étendre la fonction de recherche par des pré et post conditions puis tester le programme.

Exercice 3:

Soit la fonction C suivante.

1) Étendre cette fonction par des pré/post condition
2) Étendre cette fonction par des variants et des invariants de boucle.

```
void divisionEuclide (int a, int b){
    /*a entier naturel */
   /* b entier naturel non nul  */
   int q = 0 ;
   int r = a ;
   while (r >= b){
      q = q+1 ;
      r = r-b ;
   }
 printf ("le quaution de la division de %d par %d= %d \n", a, b, q) ;
 printf ("le reste de la division de %d par %d= %d \n", a, b, r) ;
}
```

Exercice 4:

Pour chacune de ces deux fonctions :

1) Déterminer le rôle de cette fonction.
2) Étendre cette fonction par des pré/post condition
3) Étendre cette fonction par des variants et des invariants de boucle.

float Ex (float a, int n)
 	
float A = a
int N = n
int p = 1
 	
while (N > 0)
if (N%2 == 0){
A = A*A ;
N = N//2 ;
}
else{
p = p * A;
N = N - 1 ;
}
return p ;
}
	int multiplication(int a, int b){
	/* a -- entier naturel*/
	/* b -- entier */
	int p = 0 ;
	int m = 0 ;
	
	while (m < a){
	m = m + 1 ;
	p = p + b;
	}
	
	return p ;
	}

Exercice 5:

On se propose d'écrire un programme C qui saisit un entier n (10<n<=100) puis un tableau T composé de n entiers. Le programme calcule le nombre d'éléments distincts de T. A fin d'assurer la validité, la solution proposée doit être équipée par des assertions significatives: invariants, variant, pré et post condition.

Exercice 6:

Présenter d'une façon informelle et réaliser en C un algorithme permettant de compter la fréquence des éléments stockés dans un tableau. Ces éléments sont des entiers compris entre 0 et 99. Étendre la solution par l'insertion des assertions adéquates.